Documentation développeur

Fontanie Lili

Stetten-Pigasse Romane

Talleux Inès

Sommaire

1) Objectif du projet

2) Format du projet

3) Remarques générales

4) Étape 0 : Lecture, nettoyage du fichier texte et gestion des étiquettes

4.1) Description

4.2) Notes

4.3) Utilisation des bibliothèques standard, fichiers d'entête

4.3.1) Variables globales

4.4) Fonctions du fichier nettoyage.c

5) Étape 1 : Création d'un assembleur

5.1) Description

5.2) Notes

5.3) Utilisation des bibliothèques standard, fichiers d'entête

5.4) Fonctions du fichier traduction.c

5.4.1) Fonctions utilitaires

5.4.2) Fonctions utilisées pour détecter les bêtises de l'utilisateur

5.4.3) Fonctions de la partie traduction

5.5) Traitement des erreurs

6) Étape 2 : Simulation

6.1) Description

6.2) Notes

6.3) Utilisation des bibliothèques standard, fichiers d'entête

6.4) Fonctions du fichier simulation.c

6.4.1) Fonctions *lire_memX* et *ecrire_memX*

6.4.2) Autres fonctions

7) Fonction main (de *main.c*)

7.1) Paramètres

7.2) Corps de la fonction

7.2.1) Partie nettoyage

7.2.2) Partie traduction

7.2.3) Partie simulation

8) Difficultés rencontrées

9) Pistes d'amélioration

9.1) Gérer les lignes de plus de 255 caractères dans *assembleur.txt*

9.2) Gestion plus stricte des accès mémoire

9.3) Détection des boucles infinies

9.4) Faciliter le déboggage

1) Objectif du projet

L'objectif du projet est de concevoir un outil complet de simulation de processeur, capable de :

- Lire un programme écrit en assembleur (`pgm.txt`)
- Le traduire en code machine hexadécimal (`hexa.txt`)
- Charger ce code en mémoire simulée
- Exécuter les instructions sur une machine fictive respectant l'architecture fournie
- Le projet regroupe ainsi deux parties complémentaires :
  - un assembleur
  - un simulateur de processeur

L'utilisateur interagit au lancement du programme, ainsi que lors des éventuelles instructions d'entrée `in`

2) Format du projet

Le projet prend la forme d'une archive au format ZIP contenant :

- un répertoire avec le code-source du projet organisé en plusieurs fichiers :
  - les fichiers pour la partie assembleur
  - les fichiers pour la partie simulation : le fichier `simulation.c` qui implémente la machine simulée grâce aux fonctions dont nous décrirons l'implémentation dans cette documentation ; le fichier d'entête `simulation.h` contenant les prototypes des fonctions, la définition de la structure Machine et la définition d'une constante globale et le fichier `main.c` qui est le point d'entrée du simulateur
- un sous répertoire *docs* contenant :
  - une documentation pour l'utilisateur *user.pdf*
  - la documentation développeur (que vous êtes en train de lire) *dev.pdf*, donnant des explications sur l'implémentation et indiquant quelles ont été les difficultés rencontrées au cours du projet
  - un exécutable C dont le nom est `simulateur`

3) Remarques générales

- On utilise le préfixe `const` pour certains arguments de fonction lorsque ces derniers ne doivent pas être modifiés à l'issue de l'appel à la fonction
- Certaines fonctions ont été préfixées par `static` afin de restreindre leur visibilité au fichier source dans lequel elles sont définies et d'éviter les conflits de noms
- Dans notre implémentation, les instructions sont codées sur 32 bits selon le format :

5 bits	5 bits	5bits	1 bit	16 bits
Code op.	Dest (rd)	Src1 (rn)	Imm	Src2 (rn ou #n)

- La section ***Notes*** de chaque étape sert globalement à expliquer nos choix d'implémentation

4) Étape 0 : Lecture, nettoyage du fichier texte et gestion des étiquettes

4.1) Description

Cette partie préliminaire se fait dans le fichier `nettoyage.c` qui :

- Lit le fichier texte entré en argument à l'exécution dans le terminal
- Créée une copie `copie.txt` du fichier texte dans laquelle on a retiré les espaces en début et fin de chaque ligne, vérifié la présence d'étiquettes en début de ligne, testé leur validité et supprimé ces dernières (si valides)
- `nettoyage.c` contient également des fonctions nécessaires à la gestion des sauts vers étiquettes, traitement effectué juste avant la traduction, dans le fichier `traduction.c` (*cf. étape suivante)*

→ Cette copie permet une simplification de la phase de traduction et de gestion d'erreurs de syntaxe

4.2) Notes

- Nous avons choisi de garder les lignes vides dans `copie.txt` afin de faciliter l'accès à la ligne correspondant à une erreur durant la phase de traduction
- Les seules erreurs gérées par `nettoyage.c` concernent la syntaxe et l'unicité des étiquettes,
- N'étant pas mentionnés dans l'énoncé, et ne les ayant pas vus durant le semestre, nous ne traitons pas les commentaires
- Nous avons choisi d'implémenter les étiquettes sous forme de structure `Etiquette` définie par un nom (de taille maximale 63 caractères) et une adresse

  → Le choix du nombre de caractères est arbitraire, nous avons jugé que 63 était suffisant pour des noms d'étiquettes raisonnables

  → Nous avons également choisi de limiter la taille de chaque ligne à 255 caractères car toutes les instructions valides sont largement plus petites, il n'est donc pas nécessaire de prendre plus grand, et on a une certaine marge en cas d'instruction invalide
- Afin de nous rapprocher du fonctionnement réel des assembleurs, nous interdisons les noms d'étiquettes qui :
  - correspondent à un nom de registre
  - sont composés uniquement de chiffres
  - correspondent à une adresse mémoire
- Lors du parcours, les étiquettes valides sont enregistrées sous forme de pointeur sur `Etiquette` dans un tableau global (*cf. variables globales*) on peut ainsi y accéder depuis d'autres fichiers `.c`

4.3) Utilisation des bibliothèques standard, fichiers d'entête

Module	Usage spécifique	Fonctions utilisées
`<sdtio.h>`	Affichage des erreurs, Gestion des flux (lecture et écriture de fichiers textes)	`fprintf`, `fopen`, `fgets`, `fputs`, `fseek`, `fclose`, `size_t`, `printf`
`<string.h>`	Manipuler les chaînes : lignes lues, recherche de caractères, opérations sur les noms d'étiquettes	`strlen`, `strchr`, `strcmp`, `strcpy`
`<stdlib.h>`	Allocation dynamique des structures `Etiquette` et libération mémoire de ce qui a été alloué	`malloc`, `free`, `atoi`
`<ctype.h>`	Vérification de la syntaxe des étiquettes (composées de lettres et chiffres uniquement)	`isalnum` `isdigit` `isxdigit`

`nettoyage.h`

Ce fichier d'entête est nécessaire afin :

- d'accéder au tableau dans lequel sont stockées les étiquettes valides et leur adresse correspondante depuis le fichier `traduction.c`
- d'utiliser les fonctions `estLigneVide()` `estEtiqDef()` et `trouveAdr()` dans `traduction.c` (pour l'étape de conversion de saut vers étiquette en saut vers l'adresse correspondante)
- d'utiliser `nettoyage()` et `libereEtiq()` dans le fichier `main.c`
- d'utiliser la macro suivante dans `main.c` :

```
#define TAILLE_MAX_L 256
```

4.3.1) Variables globales

On définit les variables globales suivantes dans le fichier `nettoyage.h` :

```
extern Etiquette * tab[MAX_ET];
extern size_t t_tab;
```

`tab` tableau de pointeurs sur structure `Etiquette`

`t_tab` entier désignant le nombre d'étiquettes valides stockées

4.4) Fonctions du fichier nettoyage.c

`int estLigneVide(const char * str)`

Prend une chaîne de caractère en entrée et renvoie 1 si elle "vide" et 0 sinon. On considère qu'une ligne est vide si elle est uniquement constituée d'espaces, tabulations, retour à la ligne, ou si elle contient seulement `\0`

`char * suppEspaces(char * str)`

Prend une chaîne de caractère en entrée et la renvoie sans espaces ni tabulations avant le premier caractère "non vide" (différent de `' '` et `\t`) et après le dernier caractère "non vide". Cette fonction n'est jamais appelée sur une chaîne vide.

`int estLigneEtiq(const char *str, char *nom_etiq)`

Prend deux chaînes de caractères en argument : `str` dont on veut savoir si elle possède une étiquette en début de ligne et `nom_etiq` dans laquelle la fonction met le nom de l'étiquette si elle existe. Si la fonction trouve le caractère `:` dans `str` elle considère dans un premier temps qu'il y a une étiquette. Elle effectue ensuite des vérifications sur la taille de l'étiquette et renvoie un code erreur selon le cas : renvoie 0 si l'étiquette est vide, renvoie 1 si l'étiquette est de taille valide, renvoie 2 si l'étiquette est trop longue (≥ 64 caractères sans compter le `\0` de fin de chaîne).

`int estDansTab(const char *nom)`

Prend une chaîne de caractères `nom` en argument et regarde si il existe une étiquette portant ce nom (les étiquettes sont stockées dans le tableau global `tab` *cf. variables globales*) Renvoie 1 si une étiquette a le même nom et 0 sinon.

`int estEtiqValide(const char *nom_etiq)`

Prend une chaîne de caractère représentant un. nom d'étiquette en argument et vérifie dans un premier temps que la syntaxe du nom est valide (constitué uniquement de lettres et chiffres, pas un nombre décimal ni hexadécimal, pas un nom de registre valide → r0 à r31) si la syntaxe est correcte, la fonction vérifie que l'étiquette n'est bien définie qu'une fois. Renvoie 1 si l'étiquette est valide et 0 sinon.

`char * suppEtiq(char *str)`

Prend une chaîne de caractères en argument représentant une ligne avec étiquette (cette fonction n'est jamais appelée sur une ligne sans étiquette) et la renvoie après avoir supprimé l'étiquette et le `:` ainsi que les espaces entre `:` et l'instruction

`int trouveAdr(const char *nom_etiq)`

Prend en argument une chaîne de caractères représentant un nom d'étiquette et parcours le tableau global `tab` pour trouver l'adresse mémoire associé au nom de l'étiquette. Renvoie l'adresse si trouvée et -1 sinon

`void libereEtiq(void)`

Cette fonction libère la mémoire allouée lors de la création des pointeurs sur structure `Etiquette`

`int nettoyage(int argc, char * argv)`

Fonction principale de `nettoyage.c` : elle lit le fichier texte entré en argument à l'exécution et appelle les fonctions `estLigneVide()` `suppEspaces()` `estLigneEtiq()` `estEtiqValide()` `suppEtiq()` afin de créer une copie `copie.txt` nettoyée du fichier texte (sans les espaces en début et fin de ligne ni étiquettes avant instruction). Elle vérifie la syntaxe des étiquettes en début de ligne (pas celles dans les sauts, cela se fera dans la partie suivante) et si l'ouverture et la création des fichiers textes fonctionne. Renvoie 0 si une erreur est rencontrée durant le parcours, renvoie 1 sinon.

5) Étape 1 : Création d'un assembleur

5.1) Description

Cette partie se fait dans le fichier `traduction.c` qui :

- lit le code assembleur ligne par ligne
- analyse la syntaxe de chaque instruction
- traduit chaque instruction en hexadécimal
- écrit la traduction dans `hexa.txt`

5.2) Notes

- Dans cette partie, un "**mot**" *(cf. fonction `cut`)* est défini par une suite de caractères séparée du reste de la ligne par : un espace, une virgule ou une tabulation.

  → Nous avons choisi de **limiter** le nombre de mots dans une ligne à 5 et la taille de chaque mot à 63 caractères (il faut compter le `\0`), puisqu'une instruction valide en assembleur ne dépasse pas 4 mots et qu'aucune opération ni argument ne dépasse 63 caractères *(cf. notes **étape 0**)*.

- Chaque fonction de traduction d'une instruction effectue sa propre vérification de la syntaxe de la ligne qu'elle prend en argument. Nous avons jugé cela plus simple car le nombre et la taille des arguments n'est pas le même selon l'instruction en question.

- Nous avons choisi d'utiliser le type `int` pour stocker l'instruction machine `inst` finale car le format de l'instruction nécessite 32 bits, ce qui correspond à la taille standard d'un int sur une machine comprenant une mémoire de 64 kilo-octets.

- Pour la gestion des sauts vers étiquette *(cf. fonction `traduction_jmp`)* on regarde d'abord si l'argument du saut est une étiquette valide (= présente dans le tableau global `tab` *cf. variables globales **étape 0***) on récupère l'adresse qui lui correspond à l'aide de la fonction `trouveAdr()` de `nettoyage.c` que l'on traduit comme un saut vers adresse classique. si l'argument du saut est une étiquette invalide : cela correspond au dernier cas géré par `traduction_jmp` qui renvoie une erreur.

5.3) Utilisation des bibliothèques standard, fichiers d'entête

Module	Usage spécifique	Fonctions utilisées
`<stdio.h>`	- Affichage des erreurs - Ecriture dans un autre fichier/une chaîne de caractère de sortie	`fprintf` `sprintf` `fgets` `fclose` `fopen`
`<string.h>`	Manipulation de chaînes de char	`strcmp` `memset`
`<stdlib.h>`	Conversion de chaînes en nombres	`atoi` `strtol`
`<ctype.h>`	Vérification de types	`isdigit` `isxdigit`

`traduction.h`

Ce fichier d'entête est nécessaire afin d'utiliser la fonction `traduction()` dans le fichier `main.c`

5.4) Fonctions du fichier traduction.c

5.4.1) Fonctions utilitaires

- `indice_operande(char elt[6])`

  → Renvoie l'indice de l'élément s'il est présent dans le tableau des opérandes, et renvoie -1 si l'élément n'est pas présent

- `int lecture_rkS(char *chaine, int *rk, int *S, int *imm)`

  → Cette fonction va lire et séparer `(rn)S` ou `(rd)S` pour les opérandes de types load et store. Elle gère le découpage de la parenthèse et identifie si le second terme `S` est un registre ou une valeur immédiate.

5.4.2) Fonctions utilisées pour détecter les bêtises de l'utilisateur

- `int estSuiteDeChiffres(const char *str)` / `int estHexa(const char * str)`

Fonctions de vérification qui parcourent la chaîne pour s'assurer qu'elle ne contient bien que des chiffres/caractères hexadécimaux valides.

- `int lecture_et_syntaxe_registre(char *chaine, int *reg)`

Vérifie la syntaxe d'un registre: refuse `r3x..., r, r-1` etc. Elle s'assure que le registre est bien compris entre `r0` et `r31`

- `int lecture_et_syntaxe_saut(char *chaine, int *val)`

Vérifie l'adresse d'un saut sous forme de chiffres: on refuse 3x..., -1, 70000, etc. On accepte le décimal et l'hexadécimal.

- `int lecture_et_syntaxe_imm(const char *chaine, int *val, int *imm)`

Vérifie la valeur immédiate: refuse `#3x..., #, #-1, #70000` etc. Elle distingue les immédiats décimaux et hexadécimaux et met à jour le bit `imm` .

5.4.3) Fonctions de la partie traduction

- `int cut(const char *ligne, char out[5][64])`

Cette fonction découpe une ligne du programme assembleur selon les arguments donnés et stocke les arguments dans un tableau de tableau : `out` . Elle ignore les espaces multiples et vérifie que la ponctuation est correcte (emplacements et nombre de virgules). Elle renvoie le nombre d'arguments trouvés.

Chaque fonction de la forme `traduction_X` d'une instruction a la même structure, elle prend en arguments :

- la ligne à traduire dans `buffer`
- `code_ope` est le premier argument de la ligne : il permet de savoir à quelle opération on a à faire exactement, puisqu'on ne traite pas toutes les opérations arithmétiques ou toutes les opérations de saut de la même manière
- `*out` : la chaîne de caractère dans laquelle on met la ligne traduite
- `size_out` : taille de `out`
- `nb_args` : nombre d'arguments présents dans `buffer` , indiqués par la sortie de la fonction cut

Elles suivent également la même logique : elles vérifient la validité des arguments, calculent l'instruction binaire et stockent le résultat dans out. Elles renvoient 1 en cas de succès et 0 en cas d'erreur.

Plus spécifiquement :

- `int traduction_arithmetique(char buffer[5][33], int code_ope, int size_out, char *out, int nb_args)`

Gère les opérations de type arithmétique et `rnd` et vérifie la présence de 3 registres OU 2 registres et une valeur immédiate

- `int traduction_ld(char buffer[5][33], int code_ope, int size_out, char *out, int nb_args)` / `int traduction_st(char buffer[5][33], int code_ope, int size_out, char *out, int nb_args)`

Gèrent respectivement le chargement (`ld`) et le stockage (`st`). Elles appellent la fonction `lecture_rkS` pour décoder les "mots" de type : `rn(S)` / `rd(S)`

- `int traduction_in_out(char buffer[5][33], int code_ope, int size_out, char *out, int nb_args)`

Gère les instructions d'entrée/sortie `in` et `out`

- `int traduction_jmp(char buffer[5][64], int code_ope, int size_out, char *out, int nb_args)`

Cette fonction vérifie que l'argument du saut (deuxième "mot" de la ligne : `buffer[1]`) soit valide : si l'argument est une étiquette valide elle a nécessairement été enregistrée à l'étape de nettoyage, on peut donc la trouver dans le tableau global `tab` et la remplacer par l'adresse correspondante. Si l'argument n'est pas une étiquette valide, on regarde si c'est un registre valide, sinon une adresse hexadécimale valide, sinon une adresse décimale valide. Si ce n'est aucun des cas mentionnés précédemment, l'argument est une étiquette invalide. La fonction renvoie 0 si elle a rencontré une erreur, et 1 si l'argument est valide et que la traduction fonctionne.

- `traduction(int traduction(const char *ligne, int size_out, char *out)`

→ Fonction qui orchestre la transformation d'une ligne d'assembleur en une chaîne hexadécimale du format spécifié dans l'énoncé du projet.

→ Elle appelle `cut` pour découper la ligne en mots

→ Elle récupère le code de l'opération `code_ope` via la fonction `indice_operande`

→ Grâce à un `switch` , elle appelle la bonne fonction traduction en fonction de `code_ope`

→ Elle renvoie 1 si la ligne a bien été traduite, 0 si une erreur a été détectée.

5.5) Traitement des erreurs

Nous avons défini une structure `error` contenant le message d'erreur à afficher. Nous avons ensuite initialisé au début du fichier les erreurs traitées dans ce projet.

Concrètement, lorsqu'une fonction de vérification détecte une anomalie (comme un registre > 31 ou une virgule en trop), elle appelle la fonction `PRINT_ERROR` qui affiche le message correspondant sur la sortie standard. Ensuite la fonction retourne `0` pour signaler l'échec, ce qui arrête le traitement de la ligne en cours et évite d'écrire une instruction fausse dans le fichier de sortie.

Le détail des causes possibles des erreurs est à retrouver dans la **Documentation utilisateur**.

6) Étape 2 : Simulation

6.1) Description

Cette partie se fait dans les fichiers `simulation.c` et `main.c` et suit les étapes suivantes :

Dans `simulation.c` on **implémente** le nécessaire pour pouvoir simuler mémoire et microprocesseur :

- Chargement du fichier `hexa.txt` en mémoire
- Décodage et exécution des instructions
- Calcul des flags (bits du registre d'état : `Z`, `N`, `C`)

Dans `main.c` on **lance** la simulation *(cf. Fonction main de `main.c`)*

6.2) Notes

À propos de l'implémentation du simulateur :

- Les registres sont stockés sous forme d'entiers non signés de 32 bits dont l'interprétation (signée ou non signée) dépend de l'instruction exécutée. Cela permet de gérer des adresses (non signées) et des conversions explicites lors des opérations arithmétiques.
- La mémoire est modélisée octet par octet, d'où le type `char`, pour faciliter les accès 8/16/32 bits. Les octets sont stockés en mémoire dans un tableau de caractères non signés et l'interprétation (signée ou non) s'effectue également selon l'instruction.
- Les flags `Z`, `C` et `N`, en tant qu'uniques bits logiques, sont représentés par des caractères non signés, soit un octet chacun.
- `PC` est exprimé en octets car il contient l'adresse mémoire, exprimée en octets, de la prochaine instruction à exécuter. Bien que les adresses mémoire tiennent sur 16 bits, nous avons choisi de stocker `PC` dans un entier 32 bits afin de simplifier les calculs intermédiaires et d'éviter un cast implicite lors de ceux-ci.

À propos des types `unsigned` :

- Nous avons décidé que le stockage et les calculs internes se feraient sur des types non signés pour conserver les bits et l'interprétation signée est appliquée uniquement lorsque l'instruction l'exige. (*cf. Difficultés rencontrées*)
- De même pour les registres, considérés comme suite de bits non signés pour pouvoir les manipuler directement lors des masques et décalages logiques sans propagation de bit de signe.
- Comme la mémoire adressée octet par octet ne stocke que des bits et ne possède pas de notion de signe, on utilise également le type `unsigned` pour la modéliser. Cela permet de contrôler explicitement les extensions de signe, notamment lors des opérations de chargement (*cf. fonction `execute_instruction`*) et ainsi d'éviter les extensions de signe implicites.
- `PC` représente une adresse donc n'est pas signé non plus.

À propos de l'interprétation en signé :

- Certaines instructions nécessitent une interprétation signée :
  - Valeur immédiate : étendue sur 32 bits en répétant le bit de signe (ce qui correspond à une extension de signe)
  - `out` : affiche un entier, qu'on choisit d'implémenter comme entier signé
  - Multiplication et division : `mul` doit fonctionner avec des opérandes pouvant représenter des valeurs négatives, idem pour `div`. Pour la multiplication, on ne prend que les 16 bits de poids faible de rn et S, d'où l'utilisation de masques `& 0xFFFF`. Les 16 bits de poids faible de chaque opérande sont alors interprétés comme des entiers signés (`short`), ce qui permet de gérer correctement les valeurs négatives. Un short signé sur 16 bits est dans l'intervalle [−32768 ; 32767]. Le produit maximal est donc de l'ordre de 32768×32768=2^30, ce qui tient dans un entier signé 32 bits. On peut donc calculer `prod` en `int` (32 bits) , avec un résultat sur 32 bits.

À propos de la gestion / organisation de la mémoire :

- Dans notre projet, il est nécessaire de pouvoir lire et écrire à des adresses mémoire, en particulier pour les instructions de transfert `ldb/ldh/ldw` et `stb/sth/stw`. Cela nous a amenées à faire des recherches sur l'organisation mémoire des processeurs, suites auxquelles nous avons décidé d'implémenter la méthode du *little endian*.
- En *little endian*, l'octet de poids faible d'un mot est stocké à l'adresse mémoire la plus basse, et les octets de poids plus fort sont stockés aux adresses suivantes. (*cf. fonctions* `ecrire_memX` *et* `lire_memX`)

À propos des additions / soustractions :

- L'addition est effectuée sur 64 bits et le carry (retenue) est récupéré via le bit 32.
- La soustraction est implémentée via le complément à deux : `a-b=a+(~b)+1`. On effectue un calcul intermédiaire sur 64 bits afin de préserver `C`, le bit de retenue final, qui correspond au 33ème bit du résultat.

Choix d'implémentations du switch(code_op) dans `execute_instruction` :

- *Code_op 0 à 7 : Arithmétique / Logique*

  Ces instructions calculent un résultat res à partir de `R[rn]` et `S`, l'écrivent dans `R[rd]` (si `rd != 0`), puis mettent à jour les flags.

  - Implémentation des décalages : L'instruction `shr` doit gérer des décalages positifs (à droite), des décalages négatifs (à gauche) et calculer le bit `C` comme dernier bit éjecté. Il faut donc garder S signé et ajouter des tests.
    - Si S=0, il n'y a pas de changement du contenu du registre rn.
    - Si S > 0, il y a un décalage à droite. Si S est supérieur ou égal à 32, on prend comme carry le bit de poids fort et le résultat à mettre dans rd est 0. Sinon, le dernier bit sorti correspond au bit S-1 en comptant à partir de 0, ce qu'on obtient par décalage et par l'opérateur bit à bit ET.
    - Si S<0, il y a un décalage à gauche. Pour cela, on prend l'opposé -S de S et on décale les bits vers la gauche de -S positions. Si -S>=32, le résultat est 0 mais le bit `C` correspond cette fois au bit de poids faible. Sinon le bit `C` correspond au bit 32-(-S) en comptant les bits à partir de 0.

- *Code_op 12 à 17 : Transferts mémoire*

  Ces instructions utilisent une adresse calculée :

  `addr = (base + S) & 0xFFFF;`

  où :

  - `base = R[rn]` ou `R[rd]` selon instruction
  - `S = (unsigned int) S_signed`
  - masquage : permet de conserver uniquement les 16 bits de poids faible de l'adresse calculée, ce qui revient à appliquer un modulo 65536. Ce mécanisme implémente le rebouclage de la mémoire adressée sur 64 Ko (cf. *Difficultés rencontrées/Gestion de la mémoire*).

    L'adresse étant un `unsigned int`, on cast explicitement `S_signed` en `unsigned int`. Cela implique malgré tout un recul dans la mémoire si S est négatif car dans ce cas, S est stocké en complément à 2 comme 2^32 – (-S) et ajouter 2^32 – (-S) revient à soustraire -S. Le résultat est donc valide modulo 2^32.

- *Code_op 21 à 27 : Sauts*

  Le principe est le suivant :

  - `nextPC` reçoit l'adresse cible (16 bits)
  - sinon `nextPC` reste PC+4
  - toutes les adresses cibles sont masquées : `nextPC = S_u & 0xFFFF;`

- *Code_op 28 et 29 : Entrées/sorties*

- *Code_op 30 et 31 : Divers*
  - Case 30 : rnd rd, rn, S

    Ce case produit un entier pseudo-aléatoire dans l'intervalle [a, b [avec `a = R[rn], b = S_signed` et le place dans `R[rd]` .
  - Case 31 : hlt

    L'arrêt du processeur simulé se traduit par un return 0 de la fonction `execute_instruction` .
- *Default : code opératoire inconnu*

  Si le code opératoire n'est pas reconnu, on indique un message d'erreur avec `PC` et on arrête la simulation (`return 0`).

6.3) Utilisation des bibliothèques standard, fichiers d'entête

Module	Usage spécifique	Fonctions utilisées
`<stdio.h>`	Affichage des messages, résultats (`out`) Lecture des valeurs utilisateur (`in`), Lecture des fichiers texte	`printf` , `scanf` , `FILE` , `fopen` , `fgets` , `fclose`
`<stdlib.h>`	Conversion d'une chaîne hexadécimale en valeur numérique (chargement de `hexa.txt`), Génération de valeurs pseudo-aléatoires,	`strtoul` , `rand` , `srand`
`<time.h>`	Initialisation du générateur pseudo-aléatoire (sans initialisation par une valeur variable, la fonction `rand()` produirait toujours la même séquence)	`time(NULL)`

`simulation.h`

Ce fichier d'entête permet :

- la définition de la structure `Machine` (utilisée dans `main.c`) et de la taille de la mémoire par la macro globale `TAILLE_MEM` :

```c
#define TAILLE_MEM 65536

typedef struct machine {
    unsigned int  R[32];
    unsigned int  PC;
    unsigned char Z, C, N;
    unsigned char mem[TAILLE_MEM];
} Machine;
```

→ (64 × (2^{10}) = 65536 notre mémoire fait bien 64 kilo octets)

- d'utiliser les fonctions suivantes dans `main.c` pour la simulation : `charger_hexa_txt()` , `execute_instruction()` et `machine_zero()`

6.4) Fonctions du fichier simulation.c

6.4.1) Fonctions *lire_memX* et *ecrire_memX*

Les fonctions :

`static unsigned char lire_mem8(Machine *m, unsigned int a)`

`static unsigned short lire_mem16(Machine *m, unsigned int a)`

`static unsigned int lire_mem32(Machine *m, unsigned int a)`

`static void ecrire_mem8(Machine *m, unsigned int a, unsigned char v)`

`static void ecrire_mem16(Machine *m, unsigned int a, unsigned short v)`

`static void ecrire_mem32(Machine *m, unsigned int a, unsigned int v)`

Permettent d'accéder à plusieurs cases mémoires consécutives :

- Pour reconstruire une valeur (fonctions `lire_memX`)
- Pour décomposer une valeur (fonctions `ecrire_memX`)

Exemple détaillé :

- Les fonctions `lire_memX` prennent **un pointeur sur la structure `Machine` et un entier non signé (l'adresse) comme paramètres.

```c
static unsigned short lire_mem16(Machine *m, unsigned int a) {
    a &= 0xFFFF;
    unsigned short b0 = (unsigned short)(*m).mem[a];
    unsigned short b1 = (unsigned short)(*m).mem[(a + 1) & 0xFFFF];
    return (unsigned short)(b0 | (unsigned short)(b1 << 8));
}
```

  - **Étape 1** : masquage de l'adresse pour éviter les accès hors tableau (cf. Choix du "rebouclage")
  - **Étape 2** : lecture de l'octet à l'adresse a : `b0` (poids faible)
  - **Étape 2** : lecture de l'octet à l'adresse a+1 (et masquage de l'adresse pour éviter les accès hors tableau) : `b1` (poids fort)
  - **Étape 3** : recomposition du mot 16 bits par opération bit à bit et décalage

- Les fonctions `ecrire_memX` prennent **un pointeur sur la structure `Machine` et un entier non signé (l'adresse) comme arguments, ainsi qu'un `char` , un `short` ou un `int` non signé à écrire en mémoire. Nous rappelons que les octets sont stockés en mémoire comme des caractères non signés.

```c
static void ecrire_mem16(Machine *m, unsigned int a, unsigned short v) {
    a &= 0xFFFF;
    (*m).mem[a] = (unsigned char)(v & 0xFF);
    (*m).mem[(a + 1) & 0xFFFF] = (unsigned char)((v >> 8) & 0xFF);
}
```

  - **Étape 1** : extraction de l'octet faible (`v & 0xFF`)
  - **Étape 2** : extraction de l'octet fort (`v >> 8 & 0xFF`)
  - **Étape 3** : écriture dans deux cases mémoire consécutives

6.4.2) Autres fonctions

`static int ext_signe16(unsigned short x)`

Fonction dédiée aux extensions de signe (*cf. Notes, interprétation en signé*) : prend en argument un `short` (16 bits) non signé et force son interprétation comme `short` signé, puis le cast en un `int` (32 bits) pour représenter correctement des décalages ou des opérandes immédiates négatives.

`static void set_ZN(Machine *m, unsigned int v)`

Permet la mise à jour des flags (bits du registre d'état : `Z`, `C` et `N`)

→ `C` (Carry) correspond à la retenue pour les additions et soustractions et au dernier bit éjecté pour les instructions de décalage (c'est pour cela qu'il n'apparaît pas explicitement dans la fonction)

`static void decode(unsigned int inst,int *code_op, int *rd, int *rn, int *Imm, unsigned short *src2)`

Cette fonction extrait les champs d'une instruction 32 bits en utilisant des décalages à droite pour aligner le champ sur le bit 0, puis des masques (`& 0x1F, & 0x01, & 0xFFFF`) pour ne conserver que les bits correspondant au champ (5 bits, 1 bit, 16 bits). Les champs non signés sont ensuite convertis en `int` pour éviter une conversion implicite dans le `switch` de la fonction `execute_instruction()`. Src2 est converti en `unsigned short` pour préserver exactement les 16 bits de poids faible de l'instruction, sans interprétation signée.

→ (*cf. remarques générales pour le format d'une instruction*)

`int charger_hexa_txt(Machine *m, char *c)`

Cette fonction prend en paramètre un pointeur sur la structure `Machine` et un pointeur sur une chaîne de caractères contenant le nom du fichier à charger. Elle renvoie 0 en cas d'erreur lors du chargement et 1 si le chargement est réussi.

Comme vu précédemment, le format du fichier à charger comprend une instruction par ligne codée sur 8 caractères hexadécimaux. Chaque ligne représente 32 bits. L'adresse de stockage de chaque instruction est donc calculée comme : adresse = indice_instruction × 4. Le programme est placé à partir de l'adresse 0 (variable `i` initialisée à 0)

`int execute_instruction(Machine *m)`

Fonction principale du fichier `simulation.c` elle :

- lit l'instruction à l'adresse `PC`, la décode
- construit S comme suit :
  - Si Imm=0 : src2 contient rm dans les 5 bits faibles et `S = R[rm]` ;
  - Si Imm=1 : `S = ext_signe16(src2)` (immédiat signé 16 bits étendu sur 32 bits) ;
  - L'opérande S est construit sous deux formes : une version signée (`S_signed`) utilisée pour les calculs arithmétiques et les décalages, et une version non signée (`S_u`) utilisée lorsque la valeur représente une adresse
- initialise la variable `next_PC` à l'adresse de la prochaine instruction à exécuter : par défaut, la variable `next_PC` est initialisée à l'instruction suivante
- exécute l'instruction : c'est le rôle du `switch(code_op)`
- met à jour les flags
- met à jour le `PC` avec un masque pour forcer l'adresse à rester dans l'intervalle de la mémoire (adressée de 0 à 65535)
- force r0 = 0 après chaque instruction
- retourne 0 si l'instruction est `hlt`
- retourne 1 sinon.

`void machine_zero(Machine *m)`

La fonction initialise toutes les composantes de la structure `Machine` (*cf. Utilisation des fichiers d'entête,* `simulateur.h`) à 0 pour éviter que les registres, la mémoire et les indicateurs d'état contiennent des valeurs indéfinies.

7) Fonction main (de *main.c*)

7.1) Paramètres

Les paramètres de la fonction `main` correspondent à ce qui est passé en argument à l'exécution du programme :

- `argc` : nombre d'arguments (il y en a au moins un car le nom de la commande est le premier argument)
- `argv` : tableau de chaînes de caractères, correspondant aux arguments successifs.

Dans notre projet, `argv[1]` doit contenir `pgm.txt` qui est le fichier assembleur à convertir.

7.2) Corps de la fonction

Un premier test vérifie que le programme est lancé avec le bon nombre d'arguments. En cas d'erreur, un message est affiché et l'exécution interrompue.

7.2.1) Partie nettoyage

On appelle `nettoyage(argc, argv)` de `nettoyage.c` pour générer la copie sans étiquettes à traduire. Si une erreur est rencontrée le programme s'arrête, sinon on passe à l'étape suivante.

7.2.2) Partie traduction

On ouvre `copie.txt` (*cf étape 0*) en mode lecture et on le traduit ligne par ligne avec `while (fgets(ligne, sizeof(ligne), fIn))` . On s'arrête si une erreur est rencontrée (dans ce cas l'erreur, le numéro de la ligne et la ligne originaire de l'erreur seront affichés) ou bien si on a atteint la fin de `copie.txt` (dans ce cas la traduction s'est déroulée normalement).

7.2.3) Partie simulation

On déclare la variable `m` de type `Machine` qui correspond à la machine simulée et initialise ses composantes à 0 grâce à la fonction `machine_zero` . Puis, on initialise le générateur pseudo-aléatoire nécessaire pour l'instruction `rnd` avec la commande `srand(time(NULL));` .

Le fichier `hexa.txt` est ensuite chargé en mémoire. En cas d'échec, on obtient un message d'erreur et l'arrêt immédiat du simulateur.

On lance la boucle principale de simulation qui exécute ligne par ligne `hexa.txt` avec `execute_instruction()` . La boucle se termine lorsque l'instruction `hlt` est rencontrée ou qu'une erreur est détectée dans l'exécution.

La fin de la simulation affiche l'adresse du PC final, ce qui permet de vérifier où l'exécution s'est arrêtée.

8) Difficultés rencontrées

Gestion des étiquettes dans un saut

À l'issue du nettoyage, nous voulions qu'il ne reste aucune étiquette : ni devant une ligne ni dans un saut. L'implémentation de cette idée s'est avérée compliquée, car aucune fonction de `nettoyage.c` n'analyse en détail le contenu de chaque ligne, cela se fait seulement au moment de la traduction. Nous avons pensé à utiliser la fonction `cut` de `traduction.c` dans `nettoyage.c` , mais n'avons pas trouvé de façon optimisée de l'appliquer sans répétitions (d'autre part, il aurait fallu utiliser d'autres fonctions de `traduction.c` pour pouvoir vérifier avant tout que la ligne lue a une syntaxe valide, que c'est saut, etc. avant de regarder si la destination du saut est une étiquette, et de regarder si cette dernière est valide)

Solution : nous avons décidé de séparer la gestion d'étiquettes en deux partie. Le tableau qui stocke les étiquettes avec leur adresse a été transformé en variable globale `extern` afin de pouvoir y accéder dans la partie traduction et tester la validité des étiquettes au sein d'un saut juste avant de traduire l'instruction de saut en hexadécimal. Ainsi, pas de répétition de `cut` ou autres fonctions de `traduction.c`

Gestion des cas différents dans la traduction

Dans la 1ère version, le code était beaucoup moins propre, avec beaucoup de if/else. Nous avons alors décidé d'opter pour un switch case, qui était beaucoup plus logique.

Gestion de la mémoire dans la partie simulation

Une difficulté du projet a été la gestion des adresses mémoire. Pour éviter un accès hors mémoire qui entraînerait un comportement indéfini, plusieurs approches ont été étudiées :

- Vérification systématique des bornes mémoire
- Arrêt du programme en cas d'adresse invalide
- "Rebouclement" sur 16 bits

La solution retenue est le rebouclage par masquage. En notant a l'adresse :

```
a &= 0xFFFF;
```

Ce choix permet de garantir qu'aucune adresse ne sort de l'intervalle [0 ; 65535] en conservant uniquement les 16 bits de poids faible, et de simplifier le code en évitant de nombreux tests. Cependant, cela n'empêche pas une erreur logique et n'indique pas un bug.

Choix du type unsigned

Dans l'implémentation de notre code, nous avons rencontré plusieurs difficultés qui nous ont conduites à faire le choix (*cf. notes **étape 2***): le stockage et les calculs internes se font sur des types non signés pour conserver les bits et l'interprétation signée est appliquée uniquement lorsque l'instruction l'exige.

D'abord, cela a permis de répondre à la difficulté liée à la récupération fiable de certains bits. En effet, les flags et instructions sont des informations binaires : `N` , qui est le bit 31 du résultat, le bit carry `C` , qui est le bit 32 d'un calcul élargi et les champs `code_op, rd, rn, Imm, src2` sont extraits par décalages et masques. Par exemple :

```
*code_op = (int)((inst >> 27) & 0x1F);
```

→ Décalage logique plutôt qu'arithmétique

Sans le type non signé (ici de `inst`), le décalage serait arithmétique plutôt que logique, ce qui n'est pas souhaité car on extrait des champs binaires et non des valeurs signées. Dans le décodage d'instruction, on déplace des bits sans préserver le bit de signe qui n'a aucune signification. Un décalage arithmétique modifierait les bits ajoutés à gauche, ce qui pourrait fausser l'extraction (cf. différence entre *Arithmetical Shift* et *Logical Shift* vue en cours avec les instructions `ALS` et `LLS`).

→ Récupération de la retenue après l'addition et la soustraction

Outre cette distinction entre décalage logique et arithmétique, une autre difficulté rencontrée liée à l'utilisation de types signés pour les calculs a été l'extension de signe en cas de conversion vers un type plus large, en particulier pour la récupération du bit C lors de l'addition sur 64 bits. Pour répondre à ce problème, les additions sont réalisées sur des types non signés, ce qui permet ainsi de préserver l'information contenue dans le bit 32 du résultat, qui correspond à la retenue finale.

L'interprétation signée s'effectue dans un second temps à l'aide du flag `N` qui contient le bit de poids fort du résultat. En effet, grâce au complément à deux et au fait que l'addition se fasse modulo 2^32, les bits du résultat sont les mêmes, indépendamment de l'interprétation signée ou non signée des opérandes : par le complément à 2, un nombre négatif -b se lit en non signé 2^32-b et comme l'addition se fait modulo 2^32, lorsqu'on ajoute une telle valeur, on « recule » bien de b dans l'espace des valeurs modulo 2^32.

De même pour la soustraction, l'utilisation de types non signés permet d'éviter l'extension de signe lors du passage à un type de largeur supérieure, garantissant la détection de la retenue. En raison du complément à deux, la soustraction correspond en fait à une addition : a−b=a+(~b) +1. Or, en non signé, on a : ~b+1=2^32-b ; la soustraction consiste alors à ajouter une autre valeur modulo 2^32, ce qui correspond à une addition binaire modulo 2^32. Ainsi, comme pour l'addition, grâce au complément à deux et au calcul modulo 2^32, considérer a et (~b) +1 comme non signés ne change donc rien au résultat binaire : on « recule » de b dans l'espace des valeurs modulo 2^32. Comme le calcul binaire est identique quelle que soit l'interprétation signée ou non signée des opérandes, l'interprétation du résultat peut se faire dans un second temps, en fonction du contexte d'utilisation.

Rôle des pointeurs comme paramètres dans la partie simulation

Difficulté rencontrée n°1 :

Passer un argument sans pointeur revient à passer une copie de la valeur. Or, dans notre projet, beaucoup de fonctions doivent modifier l'état de la machine via les registres, le PC, les flags et la mémoire. Nous avons donc utilisé des pointeurs pour travailler directement sur la structure `Machine` et non sur une copie.

Par exemple, sans pointeur, les modifications liées à la mise à zéro de la structure `Machine` par la fonction `machine_zero()` resteraient dans la copie et seraient perdues au retour.

Solution avec pointeur :

On utilise : `void machine_zero(Machine *m)` et on modifie directement la machine du programme ainsi :

```
Machine m;
machine_zero(&m); // &m = adresse de la Machine
```

De même, dans la fonction :

```
static void set_ZN(Machine *m, unsigned int v) {
    (*m).Z = (unsigned char)(v == 0);
    (*m).N = (unsigned char)((v >> 31) & 1);
}
```

Si `m` n'était pas un pointeur, `Z` et `N` ne seraient pas mis à jour dans la machine.

Enfin, les fonctions d'accès mémoire `lire_memX` et `ecrire_memX`, reçoivent aussi un pointeur vers la structure `Machine` pour éviter de copier complètement la mémoire de 64 Ko à chaque appel et permettre que les lectures et écritures se fasse dans la mémoire à jour et non dans une copie qui ne prendrait peut-être pas en compte des modifications de la mémoire par une autre fonction entre-temps.

Difficulté rencontrée n°2 :

En C, une fonction ne peut retourner qu'une seule valeur via `return`, mais notre fonction `decode` décrite plus haut doit renvoyer 5 résultats : code_op, rd, rn, Imm, src2.

Solution avec pointeur :

Nous devons donc passer des adresses de variables en arguments pour que `decode` les remplisse. Voilà le prototype de cette fonction : `static void decode(unsigned int,int *, int *, int *, int *, unsigned short *);`

Que nous utilisons ainsi :

```
    int code_op, rd, rn, Imm;
    unsigned short src2;
    decode(inst, &code_op, &rd, &rn, &Imm, &src2);
```

9) Pistes d'amélioration

9.1) Gérer les lignes de plus de 255 caractères dans *assembleur.txt*

Dans les fichiers `nettoyage.c` et `main.c` actuels le comportement de notre programme est indéterminé si l'utilisateur entre un fichier dont au moins l'une des lignes fait plus de 255 caractères. Comme mentionné dans la **documentation utilisateur**, une instruction valide ne dépassera jamais 255 caractères, mais si l'utilisateur commet une erreur, nous n'avons pas de certitude que celle-ci soit signalée correctement.

9.2) Gestion plus stricte des accès mémoire

Dans notre projet, les adresses mémoire sont masquées sur 16 bits :

```
addr = addr &0xFFFF;
```

Cela force toute adresse à rester dans l'espace mémoire (0–65535), mais une adresse incorrecte n'est pas signalée comme une erreur et donc un bug du programme simulé peut ne pas être détecté. Comme amélioration, nous pourrions donc ajouter une vérification explicite des bornes mémoire et afficher une erreur si une instruction tente d'accéder hors mémoire.

9.3) Détection des boucles infinies

La version actuelle peut exécuter indéfiniment un programme qui ne contient jamais `hlt`. Comme amélioration, nous pourrions ajouter un compteur d'instructions maximal, arrêter la simulation après un nombre élevé d'étapes et afficher un message d'avertissement.

9.4) Faciliter le déboggage

Dans notre projet, la simulation s'exécute sans afficher l'état interne de la machine. Nous pourrions afficher à chaque instruction le PC, l'instruction décodée, les registres modifiés et les flags pour faciliter le déboggage des programmes assembleur. Le simulateur exécuterait une instruction à la fois, afficherait l'état de la machine et attendrait que l'utilisateur appuie sur Entrée pour continuer.